Predicting S&P 500 Volatility Regimes: Milestone 1

Gradient Capital: Prisha, Mariia

Code: `github.com/mariyakoval/deep-learning` (`notebooks/milestone1.ipynb`)

1 Task and data

We try to predict how volatile the S&P 500 (the SPY ETF) will be over the next 5 trading days, specifically, a 3-class regime (calm, middle, or crisis) and the realized volatility itself. Which way the market moves is close to random, but volatility tends to be persistent, and it is what risk managers need to know.

Sources. The data in this project comes from daily SPY prices and volume from yfinance, and CPI, the 10Y–2Y Treasury spread, and the fed funds rate from FRED. We have also collected financial news (as FinBERT embeddings), but it is not used in this milestone. Each row is one trading day, and there are 4,773 of them, from 2006-01-06 to 2024-12-23.

Preprocessing. From the prices we compute the daily log return, 21-day realized volatility, the daily high-low range, and volume relative to its average over the past year (raw volume trends over time). The macro series are delayed by how long they take to be published (45 days for CPI, 35 for the fed funds rate, 1 for the spread), so the model never sees a number before it would have been available. All features are scaled using only the training data.

Labels. There are no human labels. The target, *fwd_rv*, is the standard deviation of the next 5 daily log returns, annualized. For the regimes, we split *fwd_rv* at the 50th and 87th percentiles of the training data (10.6% and 21.3%). We first planned to label regimes with a hidden Markov model, but gave it up, because its labels changed so rarely that guessing "tomorrow equals today" was right 97% of the time, and it used future data to decide each day's label.

date	log ret	rv 21	rel vol	hl range	fwd rv	regime
2020-01-02	0.0093	0.0706	−0.1034	0.0073	0.0966	calm
2020-03-16	−0.1159	0.7659	1.4734	0.0791	0.6724	crisis
2022-06-13	−0.0387	0.3170	0.7359	0.0225	0.3515	crisis

Table 1: Example rows (a calm day, the COVID crash, and the June 2022 sell-off).

Split. We split by date and never shuffle: training runs from May 2009 to 2017, validation covers 2018-2019, and the test set covers 2020-2023. The last 5 days of each split are dropped so that no target reaches into the next one, and each sample is a window of past days ending at the target day. The window can reach back before the split starts, because past features are already known when we make a prediction. The test set includes COVID, the 2022 bear market, and the SVB collapse.

split	first	last	days	calm	middle	crisis
train	2009-05-01	2017-12-21	2,178	1086	808	284
val	2018-01-02	2019-12-23	498	281	145	72
test	2020-01-02	2023-12-21	1,001	292	453	256

Table 2: Split sizes and regime counts. The test set has far more crisis days (26%) than train (13%).

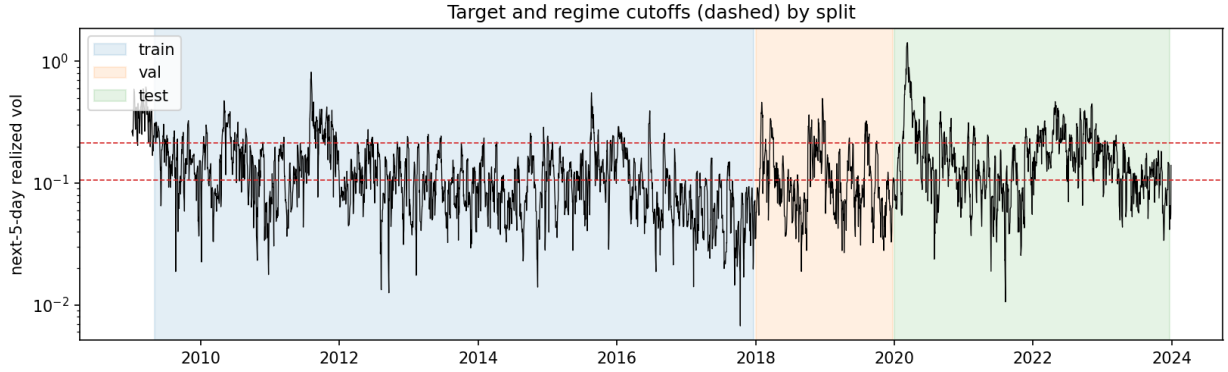


Figure 1: Target (log scale) with regime cutoffs (dashed) and the three splits.

2 Models

Baselines (0 parameters). The *majority* baseline always predicts the most common regime in the training data (calm) and the training median volatility. The *persistence* baseline assumes the next 5 days will look like the last 5. It predicts the trailing 5-day realized volatility and the regime that value falls in.

Simple model: 1D CNN (4,356 parameters). The input is the last 20 days of the 4 price features, so its shape is $(20, 4)$. It passes through two Conv1d layers (16 channels, kernel 5, ReLU, max-pool 2), then flatten, dropout 0.2, and a 32-unit linear layer with ReLU, and two output heads: 3 regime logits and one value for standardized log volatility. The loss combines class-weighted cross-entropy with MSE on log volatility. The class weights are the inverse of each class's frequency in the training data, so missing a crisis costs more. Additionally, since raw volatility is heavily right-skewed, we predict log volatility.

3 Error metric and test results

Accuracy is misleading when one class is rare, and a missed crisis is worse than a false alarm. Our main metric is macro-F1 across the three regimes, and we also report crisis recall, while for volatility we have MAE and RMSE on the annualized scale. Every model is scored on the same 1,001 test days. For the CNN, we use the mean and standard deviation over 5 seeds.

model	params	macro-F1	crisis recall	accuracy	vol MAE	vol RMSE
majority	0	0.151	0.000	0.292	0.093	0.165
persistence	0	0.543	0.574	0.541	0.077	0.116
1D CNN	4,356	0.587 ± 0.015	0.765 ± 0.023	0.581 ± 0.015	0.069 ± 0.003	0.129 ± 0.009

Table 3: Test results, 2020–2023.

The CNN beats persistence on macro-F1 (0.587 vs 0.543), crisis recall (0.765 vs 0.574), and volatility MAE (0.069 vs 0.077). It does worse on RMSE (0.129 vs 0.116), because its forecasts are smoother, so it reacts late to sudden spikes like March 2020, and RMSE penalizes those few large errors heavily (Figure 2). Training on log volatility also pulls its predictions toward typical levels. During the 2022 bear market, the CNN caught 94% of crisis days, compared with 65% for persistence.

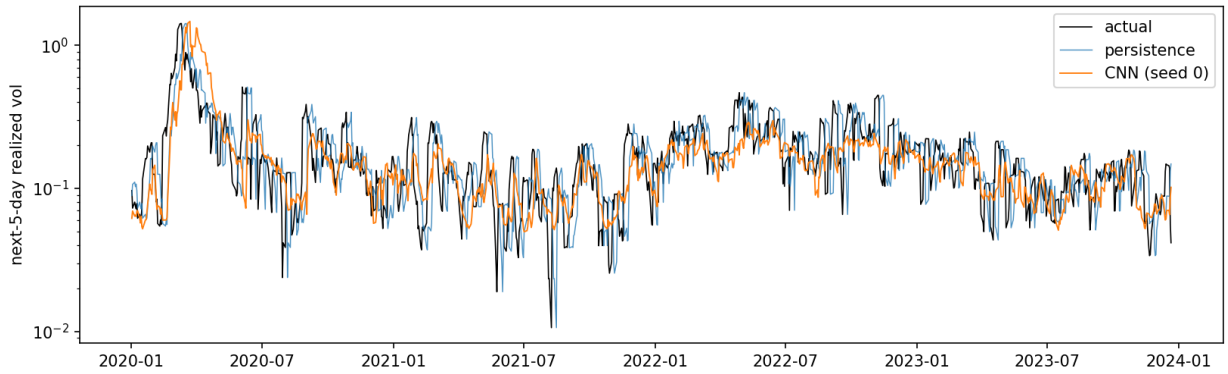


Figure 2: Test-period volatility: actual, persistence, and the CNN (seed 0).

4 Optimisation

We train with Adam (weight decay 10^{-4}), a batch size of 64, and up to 100 epochs, and stop early when validation loss has not improved for 10 epochs and keep the best weights. A grid search was performed on the validation set only, over feature set (price + macro, price, or price with relative volume), window (20, 60), channels (16, 32), and learning rate (10^{-3} or 3×10^{-4}). Each setting ran with 3 seeds, for 72 runs in total. We picked the setting with the lowest mean validation loss: price with relative volume, a 20-day window, 16 channels, and a learning rate of 0.001 (validation loss 1.363 ± 0.046).

Macro features hurt badly (best validation loss 2.12 vs 1.36), because interest rates and inflation in 2018–2023 were far outside the training range. In the test years, the fed funds rate reaches 19 training standard deviations above its training mean, so the network sees inputs it has never seen. The 20-day window also beat the 60-day window for every feature set. We have only about 2,200 training samples, and the longer window makes the linear layer larger (9–21k parameters vs 4–11k), so the model overfits sooner. Figure 3 shows validation loss flattening after 7–15 epochs while training loss keeps falling, and early stopping keeps the model at that point.

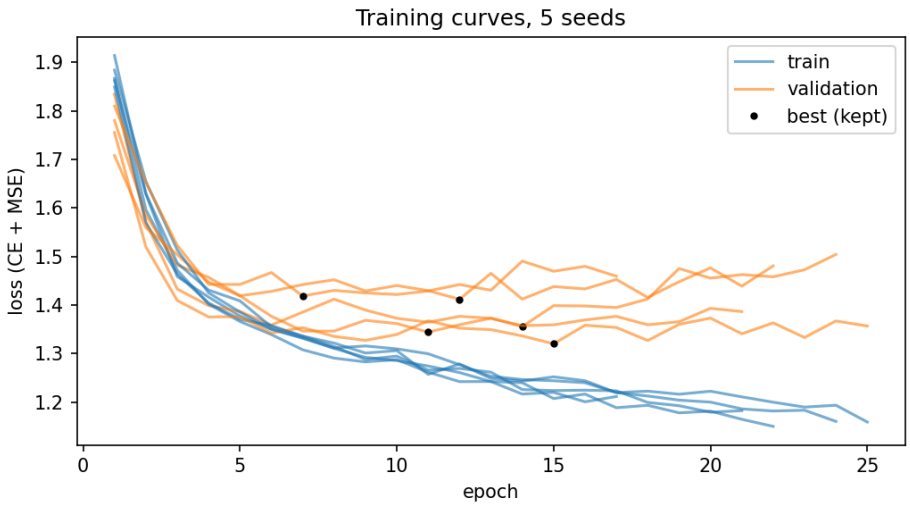


Figure 3: Training and validation loss for the selected model, 5 seeds. Dots mark the kept epoch.

Reproducibility. The notebook runs on CPU in about 5 minutes from a committed snapshot of the processed table. Seeds are fixed, PyTorch runs in deterministic mode and package versions are pinned, so different runs produce similar results.

5 What's next

Over the next weeks we will add the news branch, which is how we plan to answer our main question. We will feed the daily FinBERT embeddings (available for 98.5% of trading days in 2009–2023) into the same network and compare the result with this price-only CNN on the same test days. We will also try the macro inputs as changes rather than levels, so they stay within the training range. Finally, we will try a loss on the raw volatility scale to reduce the large errors at spikes.